

COURSE 3 - DATA ENGINEERING 2026

C3-071 - Streaming & Event-Driven Pipelines Practice Pack

Genuine-attempt tasks using the included replay/time/checkpoint simulator plus live-tool evidence only when actually available.

Artifact	C3-071
Status	2026 - LEARNER EDITION
Phase	C3-06 Streaming & Event-Driven Pipelines 13 lessons 34-46 focused hours
Prerequisite	C3-05 Gate PASS; C2B streaming/lakehouse concepts may be recalled but Kafka/Spark streaming implementation is taught here.
Current lane	Apache Kafka 4.3.1 Spark Structured Streaming 4.2.0 Delta Lake 4.4.0 Debezium 3.6.2.Final
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

House teaching rule: mental picture -> tiny event example -> guided Kafka/streaming action -> expected result -> why it worked -> break/fix -> independent changed case -> validation -> replay/state/production reasoning -> evidence. A running stream is not mastery if duplicates, late data, offsets, checkpoints or sink state are wrong.

Attempt protocol

For each task: predict -> implement/inspect -> save event/key/time/state evidence -> write confidence 1-4 and one uncertainty -> only then open matching Rxx. The simulator teaches durable reasoning but is explicitly not Kafka or Spark runtime evidence.

Starter ZIP

Extract C3_071_PRACTICE_STARTER_PROJECT_RC1.zip into your C3-06 workspace. Do not edit source fixtures or reference controls merely to make assertions pass.

P01 - Batch vs streaming and event-driven mental models

Complete the workload decision matrix for five cases. Change one latency requirement and justify the new batch/streaming choice.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R01 may be opened. Close it before the changed retry.

P02 - Kafka brokers, topics, partitions and replication

Complete the 3-partition topic/key design. Use the Kafka 4.3.1 KRaft command plan and explain why RF=1 is a laptop-only limitation.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R02 may be opened. Close it before the changed retry.

P03 - Producers, consumers and consumer groups

Draw partition assignment for 1/2/3/5 consumers in one group, then add a second independent group and explain its offsets.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R03 may be opened. Close it before the changed retry.

P04 - Offsets, retention and replay

Run the simulator on batch 01 then batch 02, save progress state, replay batch 01 and prove replay is detected rather than double-counted.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R04 may be opened. Close it before the changed retry.

P05 - Event serialization: JSON/Avro/Protobuf, schema registry and evolution

Compare event_v1.json/event_v2.json and write a compatibility decision for one additive and one breaking schema change.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R05 may be opened. Close it before the changed retry.

P06 - Kafka Connect and CDC integration patterns

Draw a Debezium/Kafka Connect CDC envelope flow including insert/update/delete, key, source position and recovery responsibility.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R06 may be opened. Close it before the changed retry.

P07 - Delivery semantics, duplicates and idempotent consumers

Prove duplicate event_id handling with the simulator and state the idempotency boundary for a downstream upsert/side effect.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R07 may be opened. Close it before the changed retry.

P08 - Event time vs processing time and late arrivals

Build an event/ingest/processing-time timeline from the supplied events and label intentionally late arrival separately from malformed time.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R08 may be opened. Close it before the changed retry.

P09 - Watermarks and windowed aggregations

Complete the watermark experiment notes. Predict accepted/late windows before running the simulator with a changed lateness threshold.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R09 may be opened. Close it before the changed retry.

P10 - Spark Structured Streaming DataFrames

Complete the Structured Streaming scaffold using explicit schema, event-time column, watermark, dedupe/window logic, sink and checkpoint placeholders. Run live only when PySpark exists.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R10 may be opened. Close it before the changed retry.

P11 - Checkpointing, restart and state recovery

Use the checkpoint contract to compare same-state restart vs fresh-state replay. Explain why random checkpoint deletion is prohibited.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R11 may be opened. Close it before the changed retry.

P12 - Streaming into Delta/lakehouse tables

Complete the Bronze/Silver streaming table contract and foreachBatch TODO with deterministic keys and separate checkpoint boundaries.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R12 may be opened. Close it before the changed retry.

P13 - Lag, operational failure modes and stream monitoring

Complete the operator dashboard/runbook for source stall, consumer lag, processing backlog, state growth and sink failure. Tie every alert to an action.

Before you touch the tool

Write: event identity -> event time -> source progress boundary -> intended sink state -> expected replay/restart behavior.

Evidence to save

- Changed code/config/diagram/runbook
- Observed simulator/command/query-progress output or explicit live-runtime PENDING note
- Event/key/count/time/state reconciliation
- Offset/checkpoint/window/lag evidence when relevant
- Confidence 1-4 + main uncertainty
- One sentence explaining why the final state is correct after replay/restart

After saving this attempt, C3-072 R13 may be opened. Close it before the changed retry.